

Clock Signals & Flip-Flops

A clock signal is used to synchronise operations and circuits together. This is used most commonly in flip flops, which store data based on the clock signal.

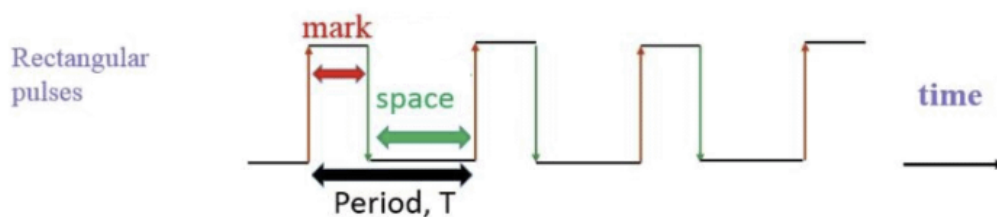
Clock Signals

A clock signal are regular/periodic HIGH pulses that oscillate at a specific frequency. The waveform of these clock signals look like square or rectangular waves.

A clock signal can be described by its mark, space, and period.

Mark refers to the period of time where the signal is HIGH, and space refers to the period of time where the signal is LOW. Period (T) refers to the time taken for one oscillation, i.e. mark + space.

Frequency can also be used to describe the signal, via $f = T^{-1}$



We can also describe the clock signal via its duty cycle, which is

$$\text{Duty Cycle} = \frac{\text{mark}}{T} \times 100\%$$

One feature of clock signals is that they have a transition between the high and low states during pulses, which is useful for storing data.

when the clock signal rises from LOW to HIGH, we call it the positive going transition (PGT), or the rising edge.

when the clock signal drops from HIGH to LOW, we call it the negative going transition (NGT), or the falling edge.

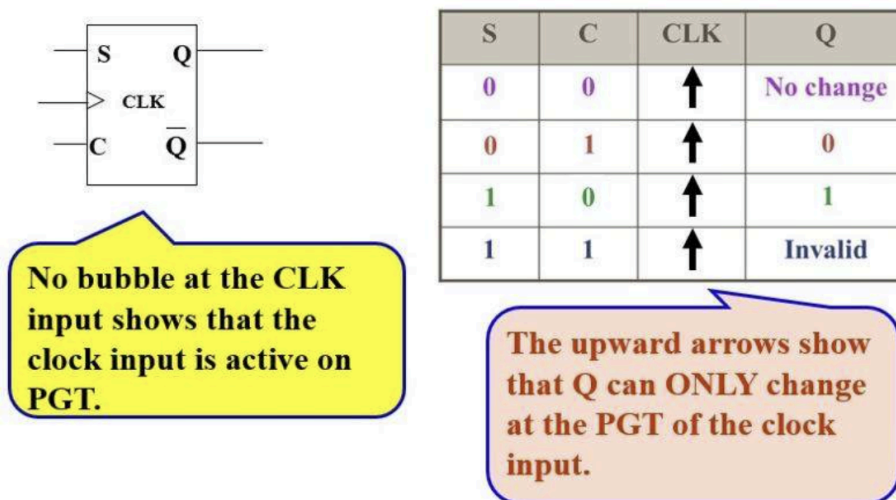
Clock signals are used in flip flops, which there are 3 common types:

Clocked SC Flip Flops

like a normal SC latch, it has a set and clear input, but will only affect the output Q and Q' when the clock signal reaches the edges.

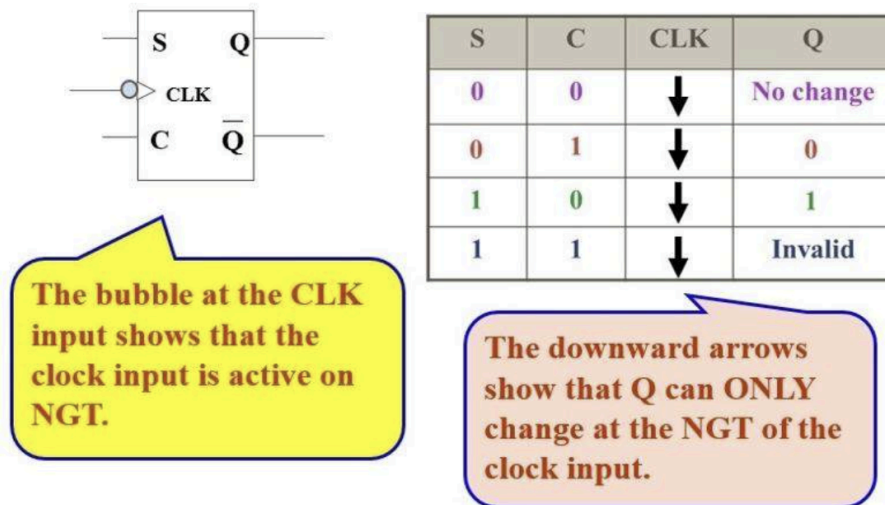
Hence, there are PGT SC latches, and NGT SC latches, which denote which edge allows the output to change.

We usually do not draw out the actual logic circuit of these flip flops, and represent it via its own symbol.



Shown is a PGT SC Flip flop; notice how the clock signal has no bubble to show that it changes due to the PGT.

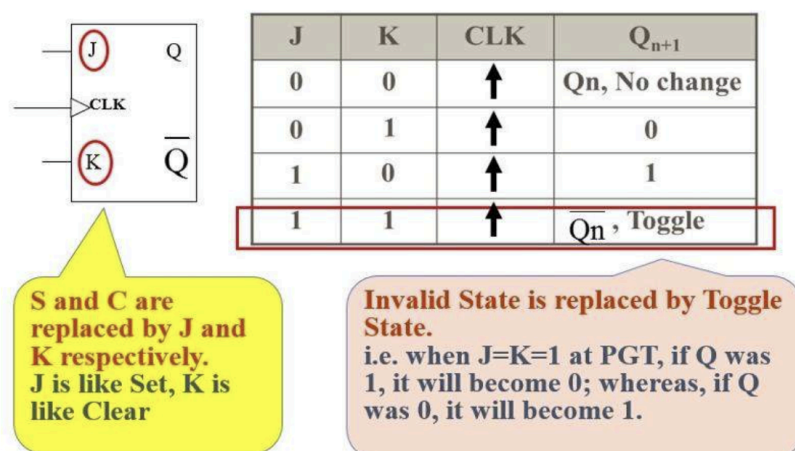
When we add a bubble (or a not gate) to the clock signal, we flip the clock signal, causing NGTs in the original signal to turn into PGTs, which when connected to the original SC flip flop, forms a NGT SC Flip Flop.



JK Flip Flops

JK flip flops are similar to SC Flip flops, however they fix the problem of invalid states in SC flip flops.

Note that Set and Clear are replaced with J and K respectively.



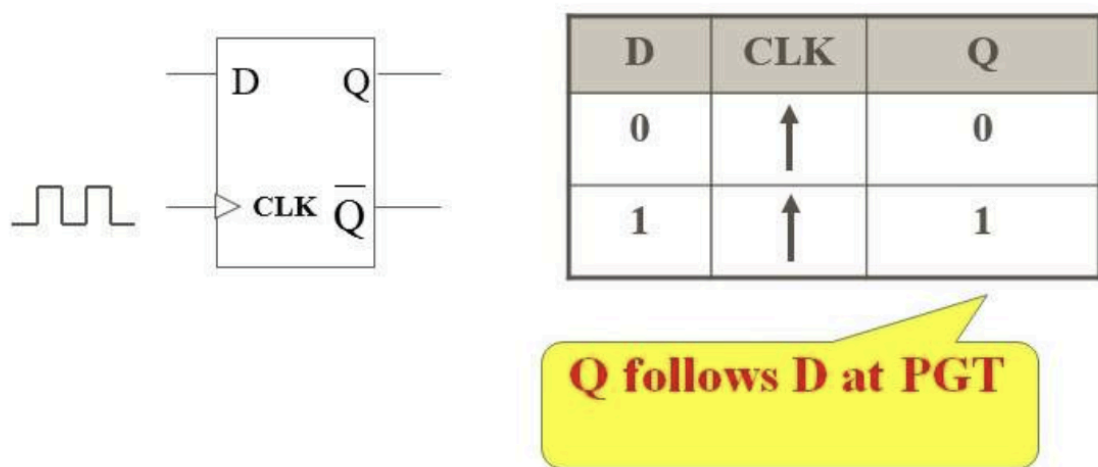
Now, the invalid state is replaced with toggle, meaning that if $Q_n = 1$, at the next clock PGT when $J = 1$ and $K = 1$, $Q_n = 0$.

This causes Q to oscillate when J and K are left as 1.

Again, shown is a PGT JK flip flop as there is no bubble at the clock signal, and the same convention applies

D Flip Flops

D (data) flip flops have only 1 input other than the clock input, as shown:



D flip flops take in a data input, but only store them as Q during the rising edge. This means it stores when D is either 1 or 0 during the PGT.

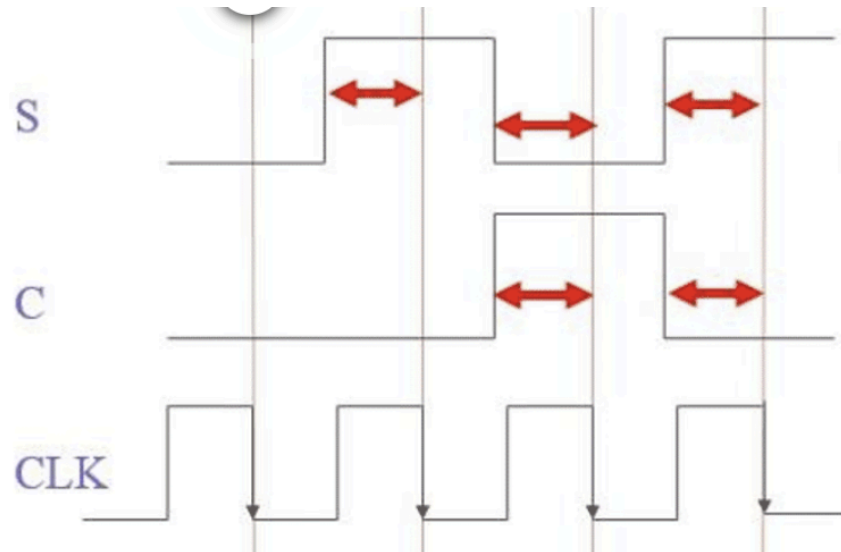
Flip-Flop Timing Considerations

As real world logic gates have some inherent delay and timing imperfections to them, they need to be considered when inputs are turned on or off.

Setup Time & Clear Time

For a flip flop to properly register the inputs, the data inputs need to be held down for sufficient time before the active clock transition (which is either PGT or NGT, depending on the type of flip flop).

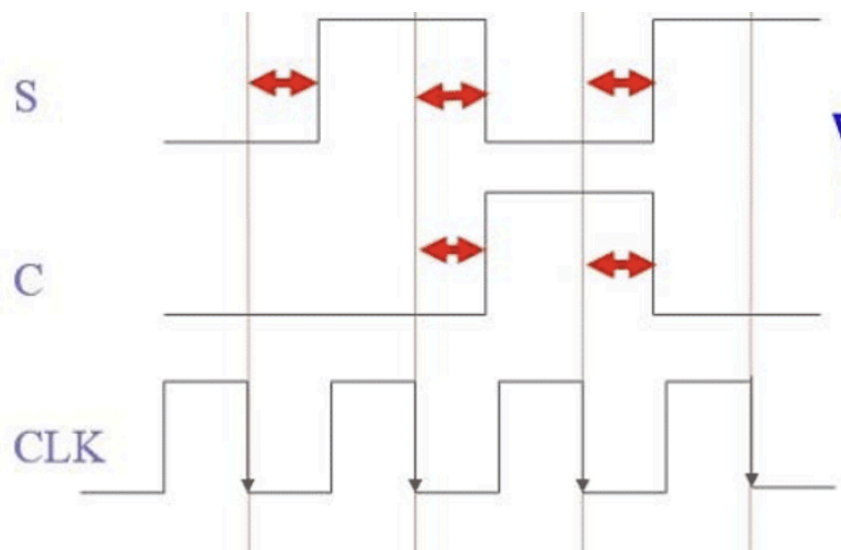
The minimum time needed to hold the inputs down until the active clock transition is called setup time, t_s . This is an inherent property and threshold of the IC chip, which when not met, may cause the flip flop to be unable to properly register the data.



Shown is the setup time for a NGT SC Flip Flop.

Likewise, the data needs to be held after the active clock transition for it to be registered too. Hence the minimum time needed for the inputs to be held down after the active clock transition is called the hold time, t_H .

Again, this is an inherent property of the IC Chip.

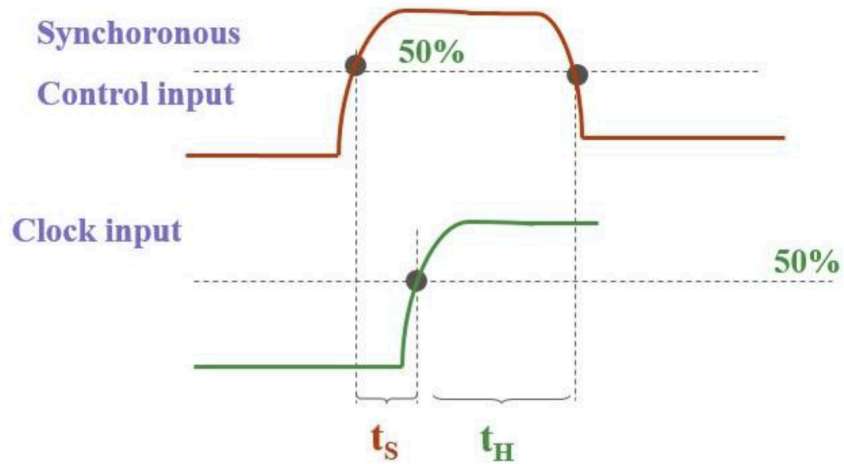


Shown is the hold time for a NGT SC Flip Flop.

As such, both the setup time threshold and hold time threshold must be met for the data input to be properly registered.

When analysing real world digital circuits, the waveforms do not have a sharp increase. Hence, we treat the point when the voltage reaches 50%

as the transition point, for both the data and the clock signal.

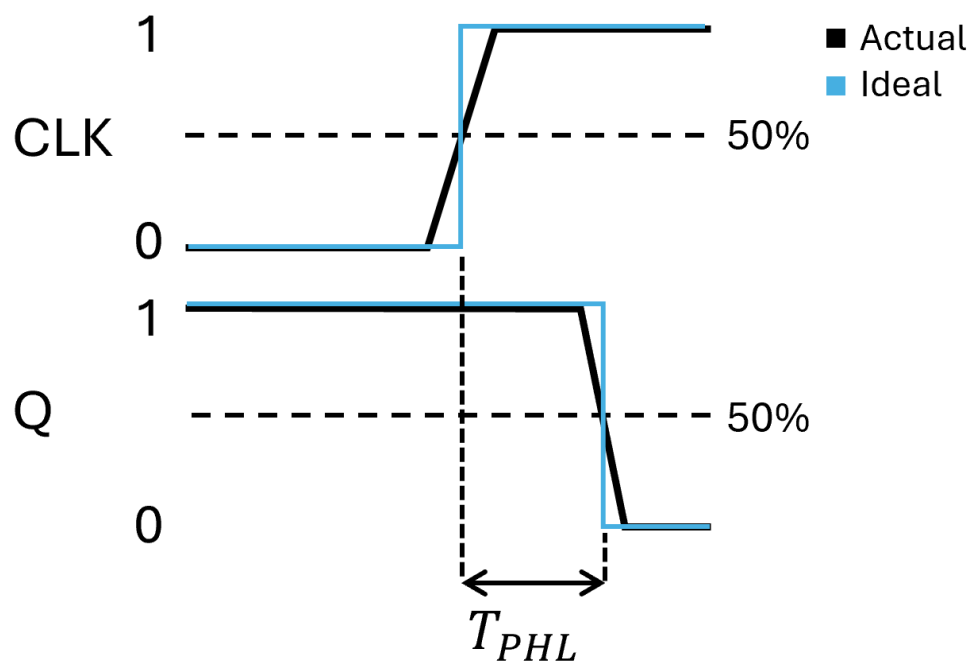
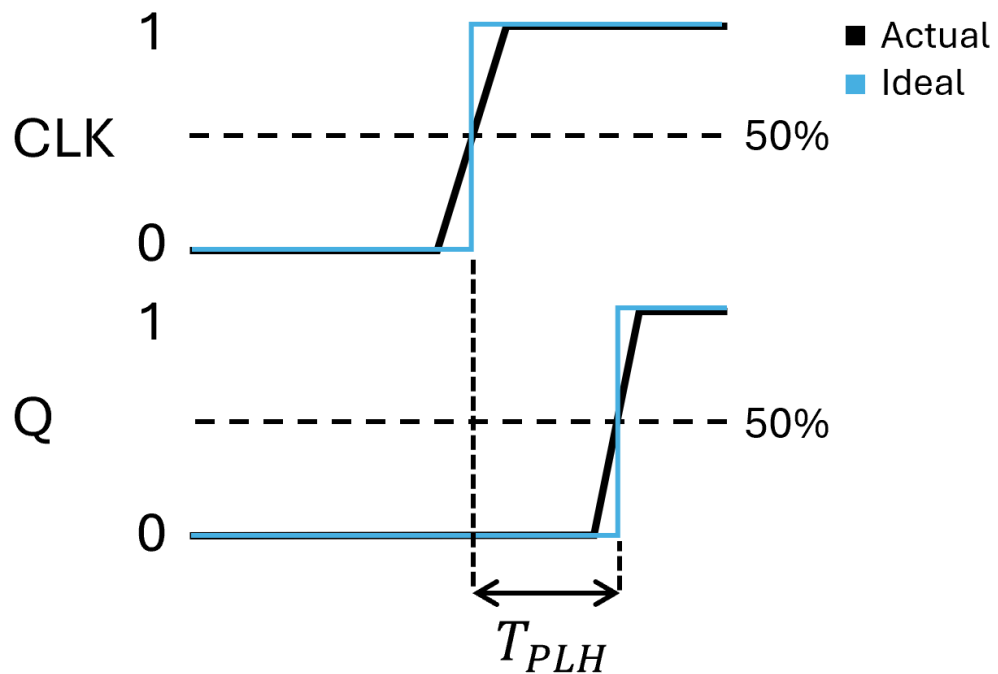


Shown is how T_{PLH} is measured, from when the CLK signal first changes, to when the Q signal changes from low to high. However, since signals cannot change immediately, we treat the point of time when it crosses the 50% mark as the change in signal, and the delay time is measured accordingly.

Propagation Delay

Propagation Delay is essentially how long the Flip flop will react to a change in the clock signal.

There are 2 kinds of propagation delay, propagation delay from low to high [T_{PLH}] and propagation delay from high to low [T_{PHL}]



Likewise, shown is how the high to low propagation delay is found.

One thing to note is that the clock signal may be PGT or NGT from when the start of the delay is measured, and does not affect the type of propagation delay measured.

Maximum Clock Frequency

Defined as the maximum frequency that can be applied to the CLK input for it to be triggered reliably. Found as:

$$f_{MAX} = \frac{1}{T_{min}}$$

The highest possible frequency means that it takes the least amount of time to make one clock cycle. Hence, T_{min} is used.

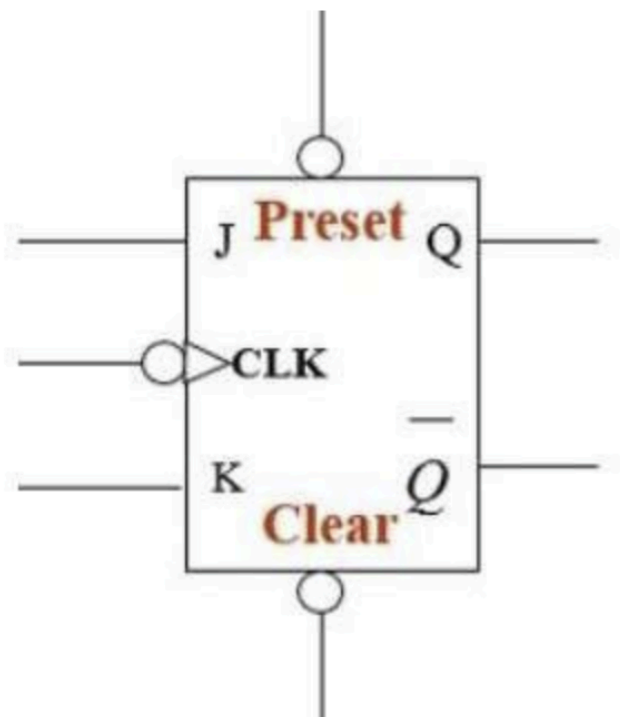
Synchronous & Asynchronous Inputs

Synchronous inputs are inputs that are synchronised with the clock signal, which is basically all the inputs we have worked with for flip flops. (S, C, D, J, K). These inputs will not reach the flip flop unless the active clock transition has reached.

Asynchronous inputs are inputs that are independent of the clock signal, and will affect the flip flop at any time they are activated. The two main ones are PRESET and CLEAR, known as asynchronous control inputs.

Asynchronous Control Inputs

Shown is a JK Flip flop with asynchronous control inputs, PRESET and CLEAR:



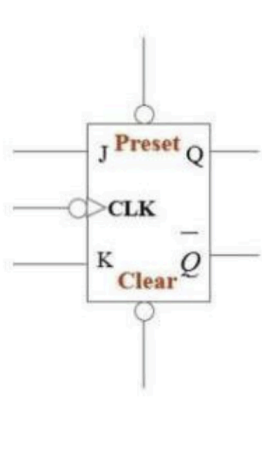
Two things to note about this:

There is no triangle at these inputs, so these are not edge triggered, but level triggered.

The bubbles indicate that these inputs are active low.

When the preset input is activated, it sets Q as 1, and Q' as 0, while when the clear input is activated, it sets Q as 0 and Q' as 1.

If both Preset and Clear are active (aka 0), the flip flop reaches an invalid state.

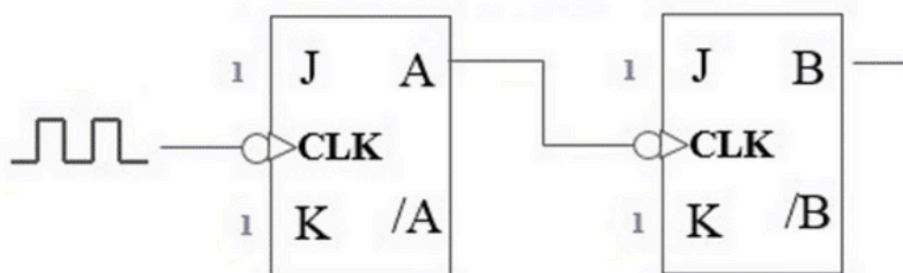


Preset	Clear	J	K	CLK	Q _{n+1}	Remarks
1	1	0	0	↓	Q _n	No change
1	1	0	1	↓	0	Clear
1	1	1	0	↓	1	Set
1	1	1	1	↓	$\overline{Q_n}$	Toggle
0	0	X	X	X	1	Invalid
0	1	X	X	X	1	Set
1	0	X	X	X	0	Clear

JK Flip Flops as Counters

Recall that JK flip flops have a toggle state, when J=K=1. The toggle state can be use to toggle each bit when counting up.

2-bit JK Flip-flop



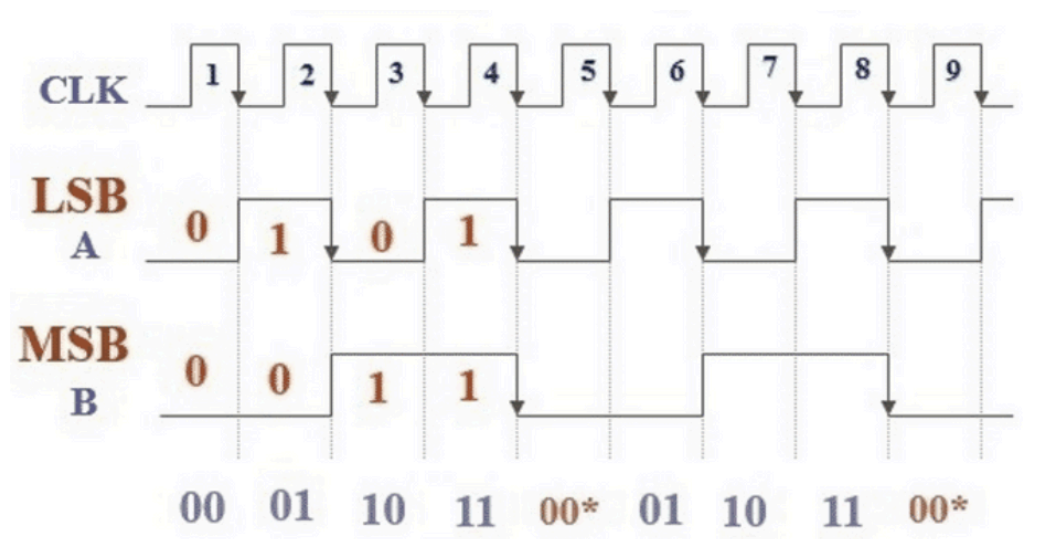
Shown is a 2 bit counter, which uses 2 NGT JK FFs, with J=K=1 to set both to the toggle state.

Hence, at every NGT of the clock signal, A will toggle at every NGT of the CLK.

As A toggles at every NGT, it toggles less frequently than the clock signal, as the NGT happens at half the frequency of when theres a clock transition.

Hence, the NGT of A will also be half as frequent as the toggle frequency of A, causing B to be a quarter of the frequency of the clock.

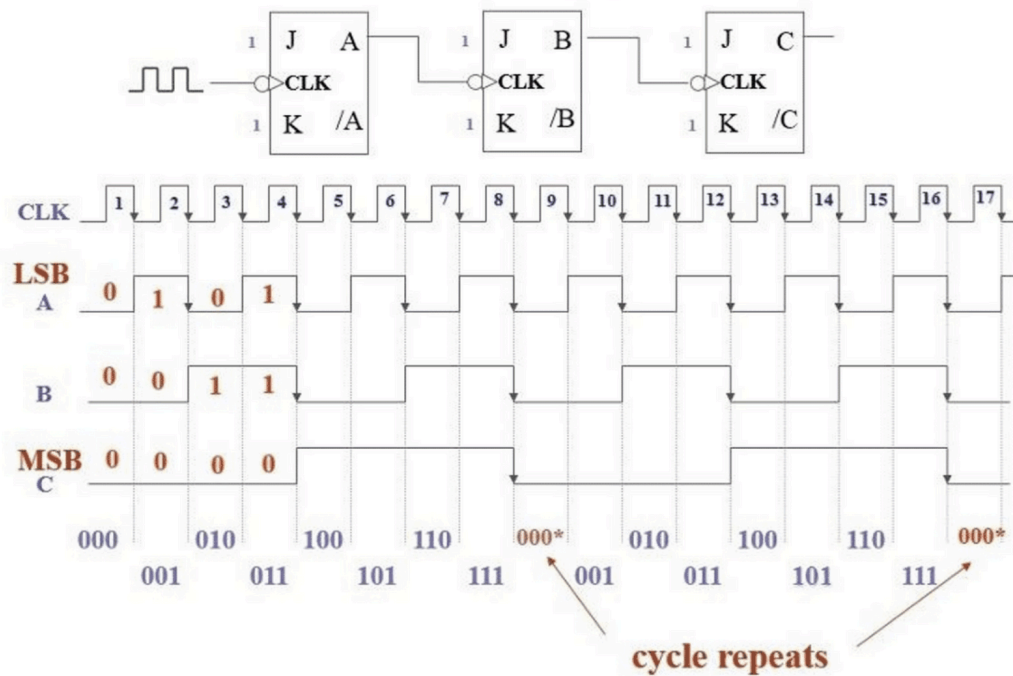
This exact 1/2 proportionality of frequency matches with how the bits change when counting up in binary:



This setup can be extended by chaining multiple NGT JK FFs together, where to make an n bit counter n JK FFs are needed.

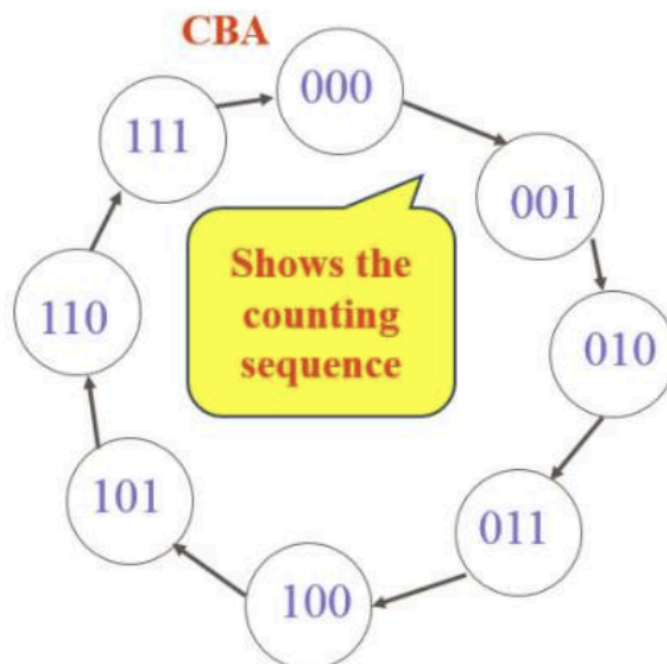
Shown is a 3 bit counter:

3-bit JK Flip-flop



Note how these counters repeat after the max value.

We can show this repetition with a state transition diagram:



This shows that after every clock pulse, which state the circuit will go to depending on the original state.

As this counter repeats, we can use MOD numbers to describe it.

A MOD number is the number of states the counter goes through before repeating. For a JK FF counter, the mod number is just 2^N , where N is the number of JK FFs used.